Principles of Artificial Intelligence and Machine Learning

Authors: Dr. Stuart Russell & Dr. Peter Norvig

Subject: Artificial Intelligence

Chapter 1: Supervised Learning and Gradient Descent

Machine learning models learn patterns directly from empirical datasets rather than following static rules.

1.1 Supervised Learning Overview:
In supervised learning, algorithms are provided input feature vectors x paired with ground-truth targets y.
When y is continuous (such as predicting house prices or temperature), the task is Regression.
When y is discrete (such as classifying spam vs non-spam emails), the task is Classification.

1.2 Gradient Descent Optimization:
Gradient descent minimizes a differentiable loss function L(w) with respect to model weights w.
The parameter update formula moves opposite to the gradient:
w_new = w_old - alpha * grad(L(w)), where alpha is the learning rate.
If alpha is too small, convergence takes excessive iterations; if too large, optimization can diverge.
Stochastic Gradient Descent (SGD) approximates the true gradient using mini-batches of training samples, offering substantial computational speedups and escaping shallow local minima.

Chapter 2: Deep Neural Networks and Backpropagation

Deep neural networks stack multiple parameterized layers to learn hierarchical abstract representations.

2.1 Neurons and Non-linear Activation Functions:
Each artificial neuron calculates an affine transformation z = w^T x + b, followed by an activation f(z).
Non-linear activations prevent deep architectures from collapsing mathematically into a single linear matrix.
Key Activation Functions:
- ReLU (Rectified Linear Unit): f(z) = max(0, z). Fast to compute, prevents vanishing gradients for positive inputs.
- Sigmoid: f(z) = 1 / (1 + exp(-z)). Compresses outputs into (0, 1), commonly used for binary probability outputs.
- Softmax: Exponentiates and normalizes output logits into a valid categorical probability distribution.

2.2 Backpropagation Algorithm:
Backpropagation computes the exact gradient of the scalar loss with respect to every weight in the network.
It applies the calculus chain rule backwards from output loss down through hidden layers to input layers.
Modern automatic differentiation engines (like PyTorch and JAX) implement backpropagation as reverse-mode AD.

Chapter 3: Generalization, Overfitting, and Regularization

A central challenge in machine learning is achieving generalization on unseen validation or test distributions.

3.1 Bias-Variance Tradeoff:
- Underfitting (High Bias): Occurs when model capacity is too restrictive to capture true data relationships.
- Overfitting (High Variance): Occurs when a high-capacity model memorizes noise and quirks of the training set.

3.2 Proven Regularization Strategies:
1. L2 Regularization (Weight Decay): Appends a penalty term lambda * ||w||^2 to the loss function.
   This penalizes large weights and promotes smoother, more generalized decision boundaries.
2. L1 Regularization (Lasso): Adds lambda * sum(|w_i|), driving small weights exactly to zero for feature selection.
3. Dropout: During each forward pass of training, randomly zeroes out hidden neuron outputs with probability p
   (typically p = 0.5), preventing fragile feature co-adaptation and mimicking an ensemble of sub-networks.
4. Early Stopping: Tracks loss on an independent validation set, terminating training when validation loss degrades.